

เอกสารสรุปและแปลเนื้อหาวิชา Software Engineering

อิงตามเอกสารการสอนของ Ian Sommerville (บทที่ 1, 2 และ 4)

ครอบคลุม: บทนำและจริยธรรม (Ch1) • กระบวนการซอฟต์แวร์ (Ch2) • วิเคราะห์ความต้องการ (Ch4)

บทที่ 1: บทนำสู่วิศวกรรมซอฟต์แวร์ (Introduction)

นิยามของวิศวกรรมซอฟต์แวร์ (Software Engineering): คือระเบียบวินัยทางวิศวกรรมที่เกี่ยวข้องกับทุกแง่มุมของการผลิตซอฟต์แวร์ ตั้งแต่ขั้นตอนแรกของการกำหนดข้อกำหนดระบบ (System Specification) ไปจนถึงการบำรุงรักษาซอฟต์แวร์หลังจากนำไปใช้งานจริง (Maintenance)

1.1 ประเด็นสำคัญและคำถามที่พบบ่อย (FAQs)

- ซอฟต์แวร์คืออะไร (What is software?):** โปรแกรมคอมพิวเตอร์และเอกสารประกอบที่เกี่ยวข้อง โดยซอฟต์แวร์อาจพัฒนาขึ้นสำหรับลูกค้าเฉพาะรายหรือเพื่อตลาดทั่วไป
- ความสำคัญทางเศรษฐกิจ:** เศรษฐกิจของประเทศที่พัฒนาแล้วทั้งหมดต้องพึ่งพาซอฟต์แวร์ โดยค่าใช้จ่ายด้านซอฟต์แวร์คิดเป็นสัดส่วนสำคัญของ GDP/GNP และต้นทุนซอฟต์แวร์มักสูงกว่าต้นทุนฮาร์ดแวร์อย่างมาก นอกจากนี้ ค่าใช้จ่ายในการดูแลรักษา (Maintenance) ยังสูงกว่าค่าใช้จ่ายในการพัฒนาตอนต้นหลายเท่า
- สาเหตุของความล้มเหลวในโครงการซอฟต์แวร์:**
 - ความซับซ้อนของระบบที่เพิ่มขึ้น: ความต้องการให้พัฒนาระบบที่ใหญ่ขึ้น ซับซ้อนขึ้น และส่งมอบเร็วขึ้น
 - การไม่ใช้วิธีการทางวิศวกรรมซอฟต์แวร์: การเขียนโปรแกรมโดยขาดระเบียบแบบแผน ส่งผลให้ระบบมีราคาแพงและมีความน่าเชื่อถือต่ำ

1.2 คุณลักษณะสำคัญของซอฟต์แวร์ที่ดี (Attributes of Good Software)

คุณลักษณะ	คำอธิบาย
1. ความสามารถในการดูแลรักษา (Maintainability)	ซอฟต์แวร์ต้องถูกเขียนขึ้นมาเพื่อให้สามารถปรับเปลี่ยนและพัฒนาต่อยอดได้ตามความต้องการของธุรกิจที่เปลี่ยนแปลงอยู่เสมอ
2. ความเชื่อถือได้และความปลอดภัย (Dependability & Security)	ครอบคลุมความน่าเชื่อถือ ความปลอดภัย และความมั่นคงปลอดภัย ระบบต้องไม่ก่อให้เกิดความเสียหายทางกายภาพหรือทางการเงินเมื่อเกิดความผิดพลาด และป้องกันการเข้าถึงจากผู้ไม่ประสงค์ดี
3. ประสิทธิภาพ (Efficiency)	ไม่ใช้ทรัพยากรระบบอย่างสิ้นเปลือง (หน่วยความจำและรอบการทำงานของซีพียู) ตอบสนองได้รวดเร็ว
4. การยอมรับจากผู้ใช้งาน (Acceptability)	ต้องเป็นที่ยอมรับของผู้ใช้งาน ใช้งานง่าย เข้าใจได้ชัดเจน และเข้ากันได้กับระบบอื่นที่ผู้ใช้งานอยู่

1.3 ประเภทของผลิตภัณฑ์ซอฟต์แวร์ (Software Products)

ผลิตภัณฑ์ทั่วไป (Generic Products)

เป็นซอฟต์แวร์ Stand-alone ที่วางจำหน่ายในท้องตลาดทั่วไปแก่ใครก็ตามที่ต้องการซื้อ เช่น โปรแกรมกราฟิก เครื่องมือบริหารโครงการ ซอฟต์แวร์ CAD โดยผู้พัฒนาซอฟต์แวร์เป็นเจ้าของสเปกและมีอำนาจตัดสินใจในการแก้ไขปรับปรุง

ผลิตภัณฑ์สั่งทำพิเศษ (Customized Products)

เป็นซอฟต์แวร์ที่พัฒนาขึ้นตามคำสั่งของลูกค้าเฉพาะราย เช่น ระบบควบคุมแบบฝังตัว ระบบควบคุมการจราจรทางอากาศ โดยลูกค้าเป็นเจ้าของสเปกและเป็นผู้กำหนดทิศทางการเปลี่ยนแปลงแก้ไข

1.4 จรรยาบรรณวิศวกรซอฟต์แวร์ (ACM/IEEE Code of Ethics)

วิศวกรซอฟต์แวร์ต้องประพฤติตนอย่างซื่อสัตย์และมีความรับผิดชอบต่อสังคมตามหลักการ 8 ประการ:

- 1. **PUBLIC (สาธารณะ):** ปฏิบัติงานโดยคำนึงถึงประโยชน์สูงสุดของสาธารณชนเป็นสำคัญ
- 2. **CLIENT AND EMPLOYER (ลูกค้าและนายจ้าง):** ปฏิบัติงานเพื่อประโยชน์สูงสุดของลูกค้าและนายจ้างภายใต้กรอบที่ไม่ขัดต่อประโยชน์สาธารณะ
- 3. **PRODUCT (ผลิตภัณฑ์):** รับผิดชอบให้ผลิตภัณฑ์และการดัดแปลงมีมาตรฐานวิชาชีพสูงสุดเท่าที่จะเป็นไปได้
- 4. **JUDGMENT (การตัดสินใจ):** รักษาความซื่อสัตย์สุจริตและความเป็นอิสระในการตัดสินใจทางวิชาชีพ
- 5. **MANAGEMENT (การจัดการ):** ผู้บริหารต้องส่งเสริมแนวทางที่มีจริยธรรมในการจัดการพัฒนาและบำรุงรักษาซอฟต์แวร์
- 6. **PROFESSION (วิชาชีพ):** ส่งเสริมชื่อเสียงและความน่าเชื่อถือของวิชาชีพ
- 7. **COLLEAGUES (เพื่อนร่วมงาน):** ปฏิบัติต่อเพื่อนร่วมงานอย่างเป็นธรรมและให้การสนับสนุนซึ่งกันและกัน
- 8. **SELF (ตนเอง):** มุ่งมั่นเรียนรู้ตลอดชีวิตในวิชาชีพและส่งเสริมการปฏิบัติตามจริยธรรม

ความรับผิดชอบทางวิชาชีพที่สำคัญ:

- **Confidentiality (การรักษาความลับ):** เคารพความลับของลูกค้า/นายจ้างแม้ไม่ได้ลงนามในสัญญา
- **Competence (ความสามารถ):** ไม่บิดเบือนระดับความสามารถของตนเอง และไม่รับงานที่อยู่นอกเหนือความสามารถของตนโดยเจตนา
- **Intellectual Property Rights (สิทธิในทรัพย์สินทางปัญญา):** เคารพกฎหมายสิทธิบัตรและลิขสิทธิ์
- **Computer Misuse (การไม่ใช้คอมพิวเตอร์ในทางที่ผิด):** ไม่นำทักษะทางเทคนิคไปใช้ทำลายระบบหรือเผยแพร่ไวรัส

บทที่ 2: กระบวนการซอฟต์แวร์ (Software Processes)

กระบวนการซอฟต์แวร์ (Software Process): ชุดของกิจกรรมที่มีโครงสร้างเพื่อใช้ในการพัฒนาระบบซอฟต์แวร์ ประกอบด้วย 4 กิจกรรมหลักพื้นฐานเสมอ ได้แก่ 1) การกำหนดความต้องการ (Specification) 2) การออกแบบและพัฒนา (Design & Implementation) 3) การตรวจสอบความถูกต้อง (Validation) 4) การวิวัฒนาการ/ปรับปรุง (Evolution)

2.1 โมเดลกระบวนการพัฒนาซอฟต์แวร์หลัก (Software Process Models)

โมเดล	แนวทางและโครงสร้าง	ข้อดี (Pros)	ข้อจำกัด (Cons)
Waterfall Model (แบบน้ำตก)	เป็นแบบ <i>Plan-driven</i> แยกแต่ละขั้นตอนชัดเจน (Requirements → Design → Implementation → Testing → Maintenance) ต้องจบทีละเฟส	• โครงสร้างชัดเจน วางแผนและควบคุมง่าย • เหมาะกับโครงการวิศวกรรมขนาดใหญ่ที่มีหลายทีมทำงานร่วมกัน	• ปรับเปลี่ยนตาม Requirement ที่เปลี่ยนไปได้ยากมาก • ลูกค้าเห็นระบบจริงช้ามากในเฟสท้ายสุด
Incremental Development (แบบเพิ่มทีละส่วน)	ผสานขั้นตอน Specification, Development, Validation เข้าด้วยกัน พัฒนาระบบออกมาทีละเวอร์ชันย่อย (Agile / Plan-driven)	• ลดค่าใช้จ่ายในการรับมือกับการเปลี่ยนแปลง • ได้รับ Feedback จากลูกค้าเร็ว • ส่งมอบชิ้นงานที่ใช้งานได้จริงอย่างรวดเร็ว	• ขาดเอกสารที่เป็นทางการของแต่ละเวอร์ชันย่อย • สถาปัตยกรรมระบบอาจเสื่อมลง (Degrade) หากขาดการทำ Refactoring สม่ำเสมอ
Reuse-oriented / Integration & Configuration	ประกอบระบบขึ้นจากส่วนประกอบเดิมหรือระบบสำเร็จรูปที่มีอยู่แล้ว (COTS - Commercial-off-the-shelf) แล้วทำการปรับแต่ง (Configuration)	• ลดต้นทุนและความเสี่ยง • ประหยัดเวลา พัฒนาและส่งมอบได้รวดเร็วมาก	• ต้องยอมประนีประนอม Requirement ให้เข้ากับซอฟต์แวร์ที่มี • ขาดการควบคุมการวิวัฒนาการของระบบในอนาคต

2.2 กิจกรรมในกระบวนการซอฟต์แวร์ (Process Activities)

- **การกำหนดความต้องการของซอฟต์แวร์ (Software Specification):** การค้นหา วิเคราะห์ และระบุวิธีการและข้อจำกัดของระบบ
- **การออกแบบและการสร้างระบบ (Software Design and Implementation):** แปลงข้อกำหนดให้กลายเป็นระบบที่ทำงานได้จริง (รวมถึง Architectural design, Database design, Interface design, Component design)
- **การตรวจสอบความถูกต้อง (Software Validation - V&V):** ยืนยันว่าระบบทำงานถูกต้องตามสเปกและตอบโจทยความต้องการของผู้ใช้ ผ่านการทดสอบ (Component testing → System testing → Acceptance testing) และการทำ V-Model
- **การวิวัฒนาการของซอฟต์แวร์ (Software Evolution):** การปรับเปลี่ยนระบบที่มีอยู่เพื่อรองรับความต้องการทางธุรกิจและสภาพแวดล้อมใหม่ๆ

2.3 การรับมือกับการเปลี่ยนแปลง (Coping with Change)

การเปลี่ยนแปลงในโครงการซอฟต์แวร์เป็นสิ่งที่หลีกเลี่ยงไม่ได้ ซึ่งนำไปสู่การทำงานซ้ำ (Rework) และต้นทุนที่เพิ่มขึ้น มี 2 กลยุทธ์หลัก:

Change Anticipation (การคาดการณ์การเปลี่ยนแปลง)

สร้าง **ต้นแบบ (System Prototyping)** อย่างรวดเร็ว เพื่อนำไปทดสอบความต้องการกับลูกค้าและตรวจสอบความเป็นไปได้ในการออกแบบก่อนเริ่มพัฒนาจริง

Change Tolerance (การยอมรับการเปลี่ยนแปลง)

ใช้แนวทาง **Incremental Delivery** ส่งมอบระบบเป็นรอบย่อยๆ เพื่อให้ผู้ใช้ได้ทดลองใช้งานจริงและสามารถปรับเปลี่ยน Requirement ในรอบถัดไปได้ง่าย

2.4 การปรับปรุงกระบวนการและระดับวุฒิภาวะ (Process Improvement & CMMI)

วงจรการปรับปรุงกระบวนการประกอบด้วย: **การวัดผล (Measure) → การวิเคราะห์ (Analyze) → การเปลี่ยนแปลง (Change)**

ระดับความสามารถของกระบวนการ (SEI Capability Maturity Model - 5 ระดับ):

- **Level 1 (Initial):** ไร้การควบคุม ไม่มีมาตรฐานชัดเจน ผลลัพธ์ขึ้นอยู่กับรายบุคคล
- **Level 2 (Managed / Repeatable):** มีการกำหนดขั้นตอนการบริหารจัดการโครงการอย่างเป็นระบบ สามารถทำซ้ำได้
- **Level 3 (Defined):** มีขั้นตอนและมาตรฐานการจัดการกระบวนการทั่วทั้งองค์กรอย่างเป็นทางการ
- **Level 4 (Quantitatively Managed):** มีการวัดผลและควบคุมคุณภาพด้วยข้อมูลเชิงปริมาณและสถิติ
- **Level 5 (Optimizing):** มุ่งเน้นการปรับปรุงและพัฒนากระบวนการอย่างต่อเนื่องโดยใช้นวัตกรรม

บทที่ 4: วิศวกรรมความต้องการ (Requirements Engineering)

วิศวกรรมความต้องการ (Requirements Engineering - RE): กระบวนการค้นหา วิเคราะห์ บันทึก และตรวจสอบความต้องการและข้อจำกัดของระบบที่ลูกค้าต้องการ เป็นรากฐานที่สำคัญที่สุด เพราะหากแก้ไขข้อผิดพลาดของความต้องการหลังส่งมอบระบบ ค่าใช้จ่ายอาจสูงกว่าการแก้ไขในตอนแรกถึง **100 เท่า**

4.1 ประเภทของความต้องการ (Types of Requirements)

User Requirements (ความต้องการของผู้ใช้)

ข้อความบรรยายด้วยภาษาธรรมชาติและแผนภาพอย่างง่าย เพื่อบอกว่าระบบจะให้บริการอะไรและมีข้อจำกัดอย่างไร เขียนขึ้นเพื่อให้ลูกค้าและผู้ใช้ทั่วไปเข้าใจได้

System Requirements (ความต้องการของระบบ)

เอกสารที่มีโครงสร้างละเอียด ระบุฟังก์ชัน บริการ และข้อจำกัดเชิงเทคนิคอย่างชัดเจน ใช้เป็นข้อตกลงและสัญญาจ้างระหว่างลูกค้ากับผู้พัฒนา

4.2 ความต้องการเชิงฟังก์ชันและไม่เชิงฟังก์ชัน

- **Functional Requirements (ความต้องการเชิงฟังก์ชัน):** ระบบทำอะไรได้บ้าง (*What the system should do*) พฤติกรรมเมื่อได้รับข้อมูลเข้า และสิ่งที่ไม่ควรทำ เช่น การค้นหารายชื่อนัดหมาย, การคำนวณปริมาณยา
- **Non-functional Requirements (ความต้องการที่ไม่ใช่ฟังก์ชัน):** ระบบข้อจำกัดและคุณสมบัติของระบบ (*Constraints & System properties*) แบ่งเป็น:
 - *Product Requirements:* ประสิทธิภาพ, ความเร็ว, พื้นที่จัดเก็บ, ความปลอดภัย, ความน่าเชื่อถือ (เช่น ระบบพร้อมทำงาน 99.9%, downtime ไม่เกิน 5 วันต่อปี/วัน)
 - *Organizational Requirements:* มาตรฐานกระบวนการ, ภาษาหรือเครื่องมือที่ต้องใช้, นโยบายองค์กร (เช่น การยืนยันตัวตนด้วยบัตรพนักงาน)
 - *External Requirements:* กฎหมาย, ข้อกำหนดความปลอดภัย, การทำงานร่วมกับระบบอื่น, กฎหมายคุ้มครองข้อมูลส่วนบุคคล (เช่น HStan-03-2006-priv)
- **Domain Requirements:** ข้อกำหนดเฉพาะทางที่มาจากบริบทของโดเมนการใช้งานนั้นๆ

4.3 ตัวชี้วัดสำหรับระบบ Non-functional Requirements (Metrics)

คุณสมบัติ (Property)	หน่วยวัดที่สามารถทดสอบได้ (Measure)
Speed (ความเร็ว)	จำนวน Transaction ที่ประมวลผลได้ต่อวินาที, เวลาตอบสนองต่อผู้ใช้, เวลา Refresh หน้าจอ
Size (ขนาด)	ขนาดความจุหน่วยความจำ (Mbytes), จำนวน ROM chips
Ease of use (ความง่ายในการใช้)	ระยะเวลาที่ต้องใช้ในการฝึกอบรม, จำนวนหน้า Help
Reliability (ความน่าเชื่อถือ)	ระยะเวลาเฉลี่ยก่อนที่ระบบจะล้มเหลว (MTTF), ความน่าจะเป็นที่ระบบจะไม่พร้อมใช้, อัตราการเกิด Failure, ความพร้อมใช้งาน (Availability)
Robustness (ความทนทาน)	เวลาที่ใช้ Restart ระบบหลังขัดข้อง, เปอร์เซ็นต์ของเหตุการณ์ที่ทำให้ระบบพัง, ความน่าจะเป็นของข้อมูลเสียหายเมื่อเกิดข้อผิดพลาด
Portability (ความสามารถในการย้ายระบบ)	สัดส่วนคำสั่งที่ขึ้นกับระบบเป้าหมาย (%), จำนวนแพลตฟอร์มปลายทางที่รองรับ

4.4 กระบวนการวิศวกรรมความต้องการ (RE Spiral Process)

กระบวนการ RE มีลักษณะเป็นวงก้นหอยวนซ้ำ (Iterative Spiral) ประกอบด้วย 4 ขั้นตอนหลัก:

1. **Requirements Discovery / Elicitation (การค้นหาคำความต้องการ):** ทำงานร่วมกับผู้มีส่วนได้ส่วนเสีย (Stakeholders) เพื่อรวบรวมข้อมูลผ่านเทคนิคต่างๆ เช่น *Interviews* (การสัมภาษณ์แบบเปิด/ปิด), *Ethnography* (การลงพื้นที่สังเกตพฤติกรรมจริง), *Scenarios & User Stories*, และ *Use Cases (UML)*
2. **Requirements Classification & Organization (การจัดหมวดหมู่และจัดกลุ่มความต้องการ):** รวบรวมความต้องการที่เกี่ยวข้องกันเข้าไว้ด้วยกัน
3. **Requirements Prioritization & Negotiation (การจัดลำดับความสำคัญและเจรจาต่อรอง):** แก้ไขข้อขัดแย้งของความต้องการจากผู้มีส่วนได้ส่วนเสียหลายฝ่าย
4. **Requirements Specification (การจัดทำเอกสารข้อกำหนด):** เขียนลงในเอกสารอย่างเป็นทางการ เช่น Software Requirements Document (SRS)

4.5 การตรวจสอบความถูกต้องและการจัดการความต้องการ

เกณฑ์ในการตรวจสอบความถูกต้องของความต้องการ (Requirements Checking):

- **Validity (ความสมเหตุสมผล):** ฟังก์ชันตอบสนองโจทย์ความต้องการของผู้ใช้จริงหรือไม่
- **Consistency (ความสอดคล้อง):** มีข้อกำหนดที่ขัดแย้งกันเองหรือไม่
- **Completeness (ความครบถ้วน):** ครอบคลุมฟังก์ชันและข้อจำกัดที่จำเป็นทั้งหมดหรือไม่
- **Realism (ความเป็นไปได้จริง):** สามารถพัฒนาได้จริงภายใต้งบประมาณ เทคโนโลยี และเวลาที่มีหรือไม่
- **Verifiability (ความสามารถในการตรวจสอบได้):** ข้อกำหนดสามารถเขียน Test Case มาวัดผลและทดสอบได้อย่างเป็นรูปธรรมหรือไม่

การจัดการการเปลี่ยนแปลงความต้องการ (Requirements Change Management):

ประกอบด้วย 3 ขั้นตอน: **1) การวิเคราะห์ปัญหาและระบุการเปลี่ยนแปลง (Problem analysis & Change spec) → 2) การวิเคราะห์ผลกระทบและประเมินต้นทุน (Change analysis & Costing)** โดยอาศัยข้อมูล Traceability → **3) การลงมือปรับเปลี่ยนเอกสารและระบบ (Change implementation)**